Register Number:192425325
Name:B.Archana

CO4-AT-3

MLA0201-Fundamentals Of Machine Learning
Comparative Analysis

QUESTION 1 – Bayesian Networks vs Markov Random Fields

Answer

A healthcare organization is developing an intelligent disease diagnosis system using patient symptoms and medical history. Two suitable probabilistic graphical models are **Bayesian Networks (BNs)** and **Markov Random Fields (MRFs)**.

1. Graph Structure

Bayesian Network:

- Uses a **directed acyclic graph (DAG)**.
- Relationships have a direction.
- Example:

Disease → Fever
Disease → Cough
Disease → Fatigue

MRF:

- Uses an **undirected graph**.
- Relationships do not have a direction.
- Example:

Disease — Fever
|
Cough

2. Conditional Independence

Bayesian Networks represent conditional independence using **d-separation**.
For example:

$$P(D, F, C) = P(D)P(F \mid D)P(C \mid D)$$

where:

- D = Disease
- F = Fever
- C = Cough

MRFs represent conditional independence using **graph separation**.

3. Inference Process

Bayesian Networks use:

- Variable elimination
- Belief propagation

- Bayesian inference
- Sampling methods

MRFs use:

- Belief propagation
- Gibbs sampling
- Markov Chain Monte Carlo
- Optimization methods

4. Uncertainty Handling

Both models can represent uncertainty.

Bayesian Networks use **conditional probability distributions**.

MRFs use **potential functions**:

$$P(X) = \frac{1}{Z}\prod_{i}\psi_i\,(X_i)$$

where $Z$is the partition function.

5. Computational Complexity

Both models can become computationally expensive when the number of variables increases.

- BN complexity depends strongly on graph structure and treewidth.
- MRFs can be expensive because calculating the partition function may be difficult.

6. Real-World Applicability

Bayesian Networks are commonly suitable for:

- Disease diagnosis
- Patient-risk prediction
- Medical decision support
- Clinical reasoning

MRFs are particularly suitable for:

- Medical image analysis
- Image segmentation
- Spatial medical data

Code:

```
# ============================================================
# QUESTION 1
# BAYESIAN NETWORK vs MARKOV RANDOM FIELD
# Healthcare Disease Diagnosis
# ============================================================

!pip -q install -U pgmpy

# ============================================================
# IMPORT LIBRARIES
# ============================================================

import numpy as np
import pandas as pd

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.models import DiscreteMarkovNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

print("Libraries imported successfully!")
```

```python
# ============================================================
# PART 1: BAYESIAN NETWORK
# ============================================================

bn_model = DiscreteBayesianNetwork([
    ("Disease", "Fever"),
    ("Disease", "Cough"),
    ("Disease", "Fatigue")
])

# Disease probability
cpd_disease = TabularCPD(
    variable="Disease",
    variable_card=2,
    values=[
        [0.70],
        [0.30]
    ],
    state_names={
        "Disease": ["No", "Yes"]
    }
)

# Fever probability
cpd_fever = TabularCPD(
    variable="Fever",
    variable_card=2,
    values=[
        [0.80, 0.20],
        [0.20, 0.80]
    ],
    evidence=["Disease"],
    evidence_card=[2],
    state_names={
        "Fever": ["No", "Yes"],
        "Disease": ["No", "Yes"]
    }
)

# Cough probability
cpd_cough = TabularCPD(
    variable="Cough",
    variable_card=2,
    values=[
        [0.75, 0.25],
        [0.25, 0.75]
    ],
    evidence=["Disease"],
    evidence_card=[2],
    state_names={
        "Cough": ["No", "Yes"],
        "Disease": ["No", "Yes"]
    }
)

# Fatigue probability
cpd_fatigue = TabularCPD(
    variable="Fatigue",
    variable_card=2,
    values=[
        [0.70, 0.30],
        [0.30, 0.70]
    ],
    evidence=["Disease"],
    evidence_card=[2],
    state_names={
        "Fatigue": ["No", "Yes"],
        "Disease": ["No", "Yes"]
    }
)

# Add CPDs
bn_model.add_cpds(
    cpd_disease,
    cpd_fever,
    cpd_cough,
    cpd_fatigue
)

# Check Bayesian Network
print("\nBayesian Network valid:")
print(bn_model.check_model())

print("\nBayesian Network edges:")
print(list(bn_model.edges()))

# ============================================================
# PART 2: BAYESIAN INFERENCE
# ============================================================

inference = VariableElimination(bn_model)

# Patient has Fever and Cough
result = inference.query(
    variables=["Disease"],
```

```
    evidence={
        "Fever": "Yes",
        "Cough": "Yes"
    }
)

print("\n========================================")
print("DISEASE DIAGNOSIS RESULT")
print("========================================")

print(result)

# ============================================================
# PART 3: DISCRETE MARKOV RANDOM FIELD
# ============================================================

mrf_model = DiscreteMarkovNetwork([
    ("Disease", "Fever"),
    ("Disease", "Cough"),
    ("Disease", "Fatigue"),
    ("Fever", "Cough")
])

print("\n========================================")
print("MARKOV RANDOM FIELD")
print("========================================")

print("MRF Nodes:")
print(list(mrf_model.nodes()))

print("\nMRF Edges:")
print(list(mrf_model.edges()))

# ============================================================
# PART 4: COMPARISON
# ============================================================

comparison = pd.DataFrame({

    "Feature": [
        "Graph Structure",
        "Direction",
        "Causality",
        "Independence",
        "Inference",
        "Uncertainty",
        "Healthcare Diagnosis"
    ],

    "Bayesian Network": [
        "Directed Acyclic Graph",
        "Directed",
        "Can represent causal relationships",
        "D-separation",
        "Variable elimination / Sampling",
        "Conditional probabilities",
        "Highly suitable"
    ],

            "Markov Random Field": [
                "Undirected Graph",
                "Undirected",
                "No natural causal direction",
                "Graph separation",
                "Belief propagation / Sampling",
                "Potential functions",
                "Moderately suitable"
            ]
        })

        print("\n========================================")
        print("MODEL COMPARISON")
        print("========================================")

        print(comparison.to_string(index=False))

        # ============================================================
        # FINAL RESULT
        # ============================================================

        print("\n========================================")
        print("FINAL RECOMMENDATION")
        print("========================================")

        print("""
        Bayesian Network is recommended for healthcare
        disease diagnosis because it can represent
        directional relationships between diseases,
        symptoms and medical history and provides
        interpretable probability estimates.
        """)
```

Output:

```
Libraries imported successfully!

...
Bayesian Network valid:
True

Bayesian Network edges:
[('Disease', 'Fever'), ('Disease', 'Cough'), ('Disease', 'Fatigue')]

========================================
DISEASE DIAGNOSIS RESULT
========================================
+--------------+----------------+
| Disease      |   phi(Disease) |
+==============+================+
| Disease(No)  |         0.1628 |
+--------------+----------------+
| Disease(Yes) |         0.8372 |
+--------------+----------------+

========================================
MARKOV RANDOM FIELD
========================================
MRF Nodes:
['Disease', 'Fever', 'Cough', 'Fatigue']

MRF Edges:
[('Disease', 'Fever'), ('Disease', 'Cough'), ('Disease', 'Fatigue'), ('Fever', 'Cough')]

========================================
MODEL COMPARISON
========================================
             Feature                  Bayesian Network           Markov Random Field
     Graph Structure            Directed Acyclic Graph              Undirected Graph
           Direction                         Directed                      Undirected
           Causality Can represent causal relationships   No natural causal direction
        Independence                      D-separation             Graph separation
           Inference    Variable elimination / Sampling Belief propagation / Sampling
         Uncertainty          Conditional probabilities           Potential functions
Healthcare Diagnosis                  Highly suitable           Moderately suitable

========================================
FINAL RECOMMENDATION
========================================

Bayesian Network is recommended for healthcare
disease diagnosis because it can represent
directional relationships between diseases,
symptoms and medical history and provides
interpretable probability estimates.
```

Comparison Table

Factor	**Bayesian Network**	**MRF**
Graph	Directed	Undirected
Causality	Can represent causality	Does not naturally represent causality
Independence	D-separation	Graph separation
Probability	Conditional probabilities	Potential functions
Inference	Variable elimination, sampling	Belief propagation, sampling
Interpretability	High	Moderate
Complexity	Can be high	Can be very high
Healthcare diagnosis	Excellent	Good
Medical image analysis	Good	Excellent

Recommendation

Bayesian Network is more appropriate for this healthcare application.
The main reasons are:

1. Disease and symptoms naturally have directional relationships.
2. It can represent medical knowledge clearly.
3. It handles uncertainty effectively.
4. It can work with incomplete patient information.
5. Its probability results are easier for doctors to interpret.

Conclusion:
For a disease diagnosis system based on **patient symptoms and medical history, Bayesian Networks are the better choice**. MRFs are more appropriate when spatial relationships, especially medical-image information, are important.

QUESTION 2 – HMM vs CRF

Answer:

An autonomous vehicle collects continuous sensor data such as speed, acceleration, steering angle, GPS and other signals. The system must recognize activities such as:

- Driving
- Turning
- Braking
- Stopping

Two important sequence models are **Hidden Markov Models (HMMs)** and **Conditional Random Fields (CRFs)**.

1. State Representation

HMM:

The actual activity is a hidden state.

Driving → Turning → Braking

The sensor measurements are observations generated from these hidden states.

CRF:

CRF directly predicts the activity labels from observed sensor features.

Sensor Data → Activity Labels

2. Observation Modeling

HMM explicitly models:

$$P(X \mid Y)$$

where X is the observation and Y is the hidden state.

CRF directly models:

$$P(Y \mid X)$$

Therefore, CRFs do not need to specify how sensor observations are generated.

3. Learning Approach

HMMs commonly use:

- Maximum likelihood

- Baum-Welch algorithm
- Expectation-Maximization

CRFs use:

- Conditional maximum likelihood
- Gradient-based optimization

4. Prediction Accuracy

HMMs work well for relatively simple temporal sequences.

CRFs can provide better performance when multiple contextual features are available.

For autonomous vehicles, activity can depend on:

- Current speed
- Previous speed
- Acceleration
- Steering
- GPS
- Previous activity

CRFs can use these features effectively.

5. Contextual Features

HMMs have stronger independence assumptions.

CRFs can use multiple overlapping and correlated features.

For example:

$$Activity_t = f(S, p, e, ed_t Acceleration_t Steering_t PreviousActivity)$$

6. Computational Efficiency

HMMs are generally computationally efficient.

CRFs require more computation during training, especially with many features.

However, CRF prediction can still be performed efficiently using dynamic programming.

7. Scalability

HMMs are easy to scale for simple models.

CRFs scale well when efficient feature representations and sufficient training data are available.

Comparison Table:

Factor	**HMM**	**CRF**
Model	Generative	Discriminative
State	Hidden state	Output label
Observation	Explicitly modeled	Not explicitly generated
Features	Limited	Rich
Context	Moderate	Strong
Training	EM / Maximum likelihood	Conditional likelihood
Prediction	Viterbi	Viterbi
Computational cost	Lower	Higher
Sequence labeling	Good	Excellent
Autonomous vehicle	Good	Very suitable

Code:

```
!pip -q install hmmlearn sklearn-crfsuite

import numpy as np
import pandas as pd

from hmmlearn import hmm
import sklearn_crfsuite
from sklearn_crfsuite import metrics

print("Libraries imported successfully!")

# Activities
activities = [
    "Driving",
    "Turning",
    "Braking"
]
X_hmm = np.array([
    [50,  1],
    [52,  1],
    [55,  2],
    [48,  0],

    [40,  0],
    [35, -1],
    [30, -1],

    [28, -2],
    [22, -3],
    [18, -2],

    [45,  1],
    [50,  2]
])

hmm_model = hmm.GaussianHMM(
    n_components=3,
    covariance_type="diag",
    n_iter=100,
    random_state=42
)

# Train HMM
hmm_model.fit(X_hmm)

# Predict hidden states
hmm_predictions = hmm_model.predict(X_hmm)

print("\n=====================================")
print("HMM PREDICTION")
print("=====================================")
```

```
print("Predicted State Numbers:")
print(hmm_predictions)

print("\nPredicted Activities:")

for state in hmm_predictions:
    print("State", state)
# Training sensor features
X_train = [

    [
        {"speed": 50, "acceleration": 1, "steering": 0},
        {"speed": 52, "acceleration": 1, "steering": 0},
        {"speed": 55, "acceleration": 2, "steering": 0}
    ],

    [
        {"speed": 40, "acceleration": 0, "steering": 20},
        {"speed": 35, "acceleration": -1, "steering": 30},
        {"speed": 30, "acceleration": -1, "steering": 35}
    ],

    [
        {"speed": 28, "acceleration": -2, "steering": 0},
        {"speed": 22, "acceleration": -3, "steering": 0},
        {"speed": 18, "acceleration": -2, "steering": 0}
    ]
]

# Correct activity labels
y_train = [

    ["Driving",
     "Driving",
     "Driving"],

    ["Turning",
     "Turning",
     "Turning"],

    ["Braking",
     "Braking",
     "Braking"]
]
crf_model = sklearn_crfsuite.CRF(
    algorithm="lbfgs",
    max_iterations=100,
    all_possible_transitions=True
)
```

```
# Train
crf_model.fit(X_train, y_train)

print("\n========================================")
print("CRF TRAINING COMPLETED")
print("========================================")
X_test = [

    [
        {"speed": 50, "acceleration": 1, "steering": 0},
        {"speed": 38, "acceleration": -1, "steering": 25},
        {"speed": 25, "acceleration": -3, "steering": 0}
    ]
]

y_test = [
    ["Driving", "Turning", "Braking"]
]
print("\n========================================")
print("CRF PREDICTION")
print("========================================")

print("Predicted Activities:")
print(crf_predictions[0])
accuracy = metrics.flat_accuracy_score(
    y_test,
    crf_predictions
)

print("\nCRF Accuracy:",
      round(accuracy * 100, 2),
      "%")

comparison = pd.DataFrame({

    "Feature": [
        "Model Type",
        "State Representation",
        "Observation Model",
```

```
            "Contextual Features",
            "Learning",
            "Prediction",
            "Computational Cost",
            "Sequence Labeling"
        ],

        "HMM": [
            "Generative",
            "Hidden states",
            "Explicit",
            "Limited",
            "Maximum likelihood / EM",
            "Viterbi",
            "Lower",
            "Good"
        ],

        "CRF": [
            "Discriminative",
            "Output labels",
            "Conditional",
            "Rich",
            "Conditional likelihood",
            "Viterbi",
            "Higher",
            "Excellent"
        ]
})

print("\n========================================")
print("HMM vs CRF COMPARISON")
print("========================================")
print(comparison.to_string(index=False))
print("\n========================================")
print("FINAL RECOMMENDATION")
print("========================================")

print("""
CRF is recommended for autonomous vehicle
activity recognition because it can combine
multiple sensor features and contextual
information for sequence labeling.
""")
```

Output:

```
WARNING:hmmlearn.base:Model is not converging.  Current: -47.98522393201962 is not greater than -47.985223871404834. Delta is -6.061478785568397e-08
Libraries imported successfully!

========================================
HMM PREDICTION
========================================
Predicted State Numbers:
[1 1 1 1 2 2 2 2 2 2 0 1]

Predicted Activities:
State 1
State 1
State 1
State 1
State 2
State 2
State 2
State 2
State 2
State 2
State 0
State 1

========================================
CRF TRAINING COMPLETED
========================================

========================================
CRF PREDICTION
========================================
Predicted Activities:
['Driving' 'Turning' 'Braking']
```

```
CRF Accuracy: 100.0 %

========================================
HMM vs CRF COMPARISON
========================================
             Feature                      HMM                    CRF
          Model Type               Generative        Discriminative
State Representation           Hidden states         Output labels
   Observation Model                Explicit           Conditional
 Contextual Features                 Limited                  Rich
            Learning Maximum likelihood / EM Conditional likelihood
          Prediction                  Viterbi                Viterbi
  Computational Cost                    Lower                 Higher
   Sequence Labeling                     Good              Excellent

========================================
FINAL RECOMMENDATION
========================================

CRF is recommended for autonomous vehicle
activity recognition because it can combine
multiple sensor features and contextual
information for sequence labeling.
```

Recommendation:
CRF is recommended for the autonomous vehicle activity-recognition system.
Reasons:

1. Multiple sensor features can be incorporated.
2. Contextual information can be used.
3. It directly predicts activity labels.
4. It is suitable for complex sequence labeling.
5. It avoids restrictive observation assumptions of standard HMMs.

Conclusion:
For a sophisticated autonomous vehicle system with multiple sensor streams, **CRF is more suitable than a traditional HMM**. HMM remains useful when the system needs a simpler and computationally lightweight sequence model.

QUESTION 3 – Bayesian Network vs MRF in Financial Risk Prediction

Answer:
A financial analytics company wants to develop an AI-based risk prediction system using customer transaction data and historical financial records.
The two possible models are:

1. **Bayesian Network**
2. **Markov Random Field**

1. Graph Representation

Bayesian Network

Uses a **directed acyclic graph**.

Example:

Income

↓

Credit Score

↓

Loan

↓

Default Risk

This represents directional relationships.

2. Dependency Modeling

Bayesian Networks represent:

$$P(X_1, X_2, .., .. X_n) = \prod_i P\left(X_i \mid Parents(X_i)\right)$$

MRFs represent:

$$P(X) = \frac{1}{Z} \prod_i \psi_i\,(X_i)$$

BNs are better when dependencies have a meaningful direction.

MRFs are useful when relationships are symmetric.

3. Inference Techniques

Bayesian Networks:

- Variable elimination
- Belief propagation
- Sampling
- Bayesian inference

MRFs:

- Belief propagation
- Gibbs sampling
- MCMC
- Optimization

4. Learning Algorithms

Bayesian Networks use:

- Maximum likelihood
- Bayesian estimation
- Structure learning
- Parameter learning

MRFs use:

- Maximum likelihood
- Gradient-based optimization

- Pseudo-likelihood
- Parameter estimation

5. Generalization

Bayesian Networks can generalize effectively when the graph structure represents domain knowledge.

For example:

Income

↓

Debt

↓

Default Risk

This allows the system to estimate risk for new customers.

MRFs can also generalize, but performance depends on the learned graph and potential functions.

6. Computational Requirements

Both models can become computationally expensive.

Bayesian Network

Complexity depends on:

- Number of variables
- Network structure
- Treewidth
- Number of states

MRF

Complexity depends on:

- Number of variables
- Graph connectivity
- Partition function
- Inference algorithm

Dense MRFs can become particularly expensive.

7. Suitability for Financial Risk Prediction

Bayesian Network

Very suitable for:

- Credit risk
- Loan default prediction
- Customer risk
- Financial decision support
- Fraud probability

MRF

Useful for:

- Transaction networks
- Customer relationships
- Fraud networks

- Interconnected financial entities

Code:

```
# QUESTION 3
# BAYESIAN NETWORK vs MARKOV RANDOM FIELD
# FINANCIAL RISK PREDICTION
# ============================================================

# Install required library
!pip -q install -U pgmpy
import pandas as pd
import numpy as np

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.models import DiscreteMarkovNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

print("Libraries imported successfully!")

bn = DiscreteBayesianNetwork([
    ("Income", "CreditScore"),
    ("CreditScore", "DefaultRisk"),
    ("Debt", "DefaultRisk"),
    ("Transactions", "DefaultRisk")
])
cpd_income = TabularCPD(
    variable="Income",
    variable_card=2,
    values=np.array([
        [0.60],
        [0.40]
    ]),
    state_names={
        "Income": ["Low", "High"]
    }
)
cpd_debt = TabularCPD(
    variable="Debt",
    variable_card=2,
    values=np.array([
        [0.55],
        [0.45]
    ]),
    state_names={
        "Debt": ["Low", "High"]
    }
)

cpd_transactions = TabularCPD(
    variable="Transactions",
    variable_card=2,
    values=np.array([
        [0.70],
        [0.30]
    ]),
    state_names={
        "Transactions": ["Normal", "Risky"]
    }
)

cpd_credit = TabularCPD(
    variable="CreditScore",
    variable_card=2,
    values=np.array([
        [0.75, 0.30],
        [0.25, 0.70]
    ]),
    evidence=["Income"],
    evidence_card=[2],
    state_names={
        "CreditScore": ["Poor", "Good"],
        "Income": ["Low", "High"]
    }
)
```

```
cpd_default = TabularCPD(
    variable="DefaultRisk",
    variable_card=2,
    values=np.array([
        [0.90, 0.70, 0.75, 0.50, 0.80, 0.60, 0.65, 0.40],
        [0.10, 0.30, 0.25, 0.50, 0.20, 0.40, 0.35, 0.60]
    ]),
    evidence=[
        "CreditScore",
        "Debt",
        "Transactions"
    ],
    evidence_card=[2, 2, 2],
    state_names={
        "DefaultRisk": ["Low", "High"],
        "CreditScore": ["Poor", "Good"],
        "Debt": ["Low", "High"],
        "Transactions": ["Normal", "Risky"]
    }
)
bn.add_cpds(
    cpd_income,
    cpd_debt,
    cpd_transactions,
    cpd_credit,
    cpd_default
)
print("\n========================================")
print("BAYESIAN NETWORK")
print("========================================")

print("Model valid:", bn.check_model())

print("\nNetwork Edges:")
print(list(bn.edges()))

inference = VariableElimination(bn)

result = inference.query(
    variables=["DefaultRisk"],
    evidence={
        "Income": "Low",
        "CreditScore": "Poor",
        "Debt": "High",
        "Transactions": "Risky"
    }
)

print("\n========================================")
print("FINANCIAL RISK PREDICTION")
print("========================================")
```

```
print(result)
mrf = DiscreteMarkovNetwork([
    ("Income", "CreditScore"),
    ("CreditScore", "Debt"),
    ("Debt", "DefaultRisk"),
    ("Transactions", "DefaultRisk"),
    ("CreditScore", "DefaultRisk")
])


print("\n========================================")
print("MARKOV RANDOM FIELD")
print("========================================")

print("MRF Nodes:")
print(list(mrf.nodes()))

print("\nMRF Edges:")
print(list(mrf.edges()))

comparison = pd.DataFrame({
```

```
            "Graph Representation",
            "Dependency",
            "Causality",
            "Inference",
            "Learning",
            "Interpretability",
            "Missing Data",
            "Financial Risk Prediction"
        ],

        "Bayesian Network": [
            "Directed DAG",
            "Directional",
            "Strong",
            "Variable elimination / Sampling",
            "Structure + Parameter learning",
            "High",
            "Good",
            "Highly suitable"
        ],

        "MRF": [
            "Undirected graph",
            "Symmetric",
            "Weak",
            "Belief propagation / Sampling",
            "Parameter learning",
            "Moderate",
            "Good",
            "Suitable"
        ]
    })


    print("\n========================================")
    print("MODEL COMPARISON")
    print("========================================")

    print(comparison.to_string(index=False))
    print("\n========================================")
    print("FINAL RECOMMENDATION")
    print("========================================")
    print("""
    Bayesian Network is recommended for general
    financial risk prediction because it represents
    directional relationships, handles uncertainty,
    supports missing information and provides
    interpretable probability estimates.
    MRFs are particularly useful for interconnected
    transaction and fraud networks.
    """)
```

Output:

```
Libraries imported successfully!

========================================
BAYESIAN NETWORK
========================================
Model valid: True

Network Edges:
[('Income', 'CreditScore'), ('CreditScore', 'DefaultRisk'), ('Debt', 'DefaultRisk'), ('Transactions', 'DefaultRisk')]

========================================
FINANCIAL RISK PREDICTION
========================================
+-------------------+--------------------+
| DefaultRisk       |   phi(DefaultRisk) |
+===================+====================+
| DefaultRisk(Low)  |             0.5000 |
+-------------------+--------------------+
| DefaultRisk(High) |             0.5000 |
+-------------------+--------------------+

========================================
MARKOV RANDOM FIELD
========================================
MRF Nodes:
['Income', 'CreditScore', 'Debt', 'DefaultRisk', 'Transactions']

MRF Edges:
[('Income', 'CreditScore'), ('CreditScore', 'Debt'), ('CreditScore', 'DefaultRisk'), ('Debt', 'DefaultRisk'), ('DefaultRisk', 'Transactions')]
```

```
======================================
MODEL COMPARISON
======================================
                  Factor               Bayesian Network                        MRF
     Graph Representation                   Directed DAG             Undirected graph
              Dependency                    Directional                    Symmetric
               Causality                         Strong                        Weak
               Inference Variable elimination / Sampling Belief propagation / Sampling
                Learning  Structure + Parameter learning           Parameter learning
        Interpretability                           High                     Moderate
            Missing Data                           Good                         Good
Financial Risk Prediction                 Highly suitable                     Suitable

======================================
FINAL RECOMMENDATION
======================================

Bayesian Network is recommended for general
financial risk prediction because it represents
directional relationships, handles uncertainty,
supports missing information and provides
interpretable probability estimates.

MRFs are particularly useful for interconnected
transaction and fraud networks.
```

Comparison:

Factor	**Bayesian Network**	**MRF**
Graph	Directed	Undirected
Causal relationships	Strong	Weak
Dependency	Directional	Symmetric
Inference	Variable elimination	Belief propagation
Learning	Structure + parameters	Parameter estimation
Interpretability	High	Moderate
Missing data	Good	Good
Computational cost	Moderate–High	High
Risk prediction	Excellent	Good
Fraud networks	Good	Excellent

Recommendation:

Bayesian Network is recommended for general financial risk prediction.

Reasons:

1. Financial relationships often have meaningful direction.
2. It provides interpretable probabilities.
3. It can incorporate expert financial knowledge.
4. It can work with incomplete information.
5. It is suitable for risk-based decision-making.
6. It can calculate probabilities such as:

$$P(D, e, f, ault \mid IncomeCreditScoreDebtTransactions)$$

Conclusion:

For general financial risk prediction, Bayesian Networks are more effective because they provide interpretable directional dependencies and probabilistic risk estimates.

However, MRFs can be preferred for highly interconnected transaction or fraud networks where relationships are naturally undirected.